WHY YOU ARE READING THIS

Symptom: the SYS TEC sniffer decodes 0x370 EPAS_sysStatus correctly, the rack reports back its measured angle, BUT eacStatus stays at **INHIBITED** and the wheel does not move when move.py or the GUI sends 0x488 commands.

What this means: the rack is hearing us but its internal "am I allowed to steer?" gate is returning false. On a stock pre-AP rack, that gate reads GTW_epasControlType (0x101) and EPB_epasEACAllow (0x214) -- both always 0 from the stock GTW and EPB modules. The gregjhogan / BogGyver patch replaces those reads with hardcoded 1s. If we are stuck at INHIBITED with our controlType=1 frames flying, the patch did not actually take.

This document covers: (1) re-running the BogGyver flasher via the comma 3X, (2) verifying the patch landed, (3) bench testing afterward, and (4) a troubleshooting matrix.

PRE-FLIGHT CHECKLIST (do not skip)

Check	Why
Front wheels OFF the ground (jack stands or lift)	Patcher reboots rack in/out of bootloader; rack motor may twitch.
12V battery voltage ≥ 12.4V at rest	Brown-out mid-flash bricks the rack. Worst case scenario.
Tesla in Drive Ready (key in, brake pressed)	Rack needs full power, not Accessory.
Comma 3X powered on, booted into BogGyver build	Flasher is bundled in BogGyver's openpilot tree only.
Helper in driver seat to PRESS AND HOLD brake pedal	Releases would let EPB module hijack UDS session and corrupt flash.
3X has SSH enabled (for command-line fallback)	If UI button fails silently, you'll need ssh comma@192.168.43.1
Backup of original firmware exists on 3X (.bin file)	Lets us restore if something goes wrong. See Step 1 below.

STEP 1 — DIAGNOSE FIRST (before reflashing)

Reflashing without diagnosing wastes time. SSH into the 3X and check what state the previous flash attempt left things in.

1.1 — Connect the 3X to your phone hotspot OR plug it into your laptop USB. SSH in:

```
ssh comma@192.168.43.1
```

Password is 'comma' on a stock build. If it fails, see troubleshooting page 4.

1.2 — Check the firmware backup file:

```
cd /data/openpilot/selfdrive/car/modules/teslaEpasFlasher
ls -la *.bin
md5sum epas-firmware-0x7000-0x45fff.bin
```

Three possible MD5 outcomes (write down which you got):

Result	What it means	Next step
MD5 = 9e51ddd80606fbdaaf604c73c8dde0d1	Backup is unpatched stock firmware. Last flash extracted bytes but did not write the patch.	Reflash. (Step 2.)
MD5 = anything else	Backup is non-stock firmware. Either patched, or your rack shipped with a different stock version.	Run verify-patch script (Step 3) before reflashing.
.bin file does not exist	Flash never even started reading. CAN connection failed.	Check OBD-II wiring then reflash.

STEP 2 — RUN THE FLASHER

Two ways to run the patcher. Path A (UI button) is what BogGyver's docs recommend. Path B (shell) gives you a log we can debug with if it fails again. Use Path B if Path A failed last time silently.

Path A — Comma 3X UI button

2A.1 — On the 3X screen, go to Settings.

2A.2 — Open the Tesla / BogGyver settings panel. Scroll until you find a button labeled **"Flash EPAS"** (sometimes seen as "Patch EPAS"). Source: tsettings.cc line 264 in BogGyver's repo.

2A.3 — Helper presses brake pedal and HOLDS. Tap **Flash EPAS**.

2A.4 — A black terminal pops up showing flash progress. Watch for these milestones:

- "start extended diagnostic session ..."
- "request security access seed ..."
- "extract firmware ..." with progress bar (~1 min)
- "patch firmware ..." listing 3 byte changes
- "flash bootloader ..." then "transfer data ..." (~2 min)
- "reset ..." then "complete!"

2A.5 — When you see **"FLASH PROCESS COMPLETED"**, helper releases the brake. Reboot the 3X.

If you see "FLASH PROCESS FAILED" at any point: STOP. Do NOT reboot the car or attempt to drive. Take a phone photo of the entire terminal output and call Derek.

Path B — SSH command line (logged)

2B.1 — From your laptop, SSH into the 3X (same as 1.1):

```
ssh comma@192.168.43.1
```

2B.2 — Helper presses brake. Run the flasher with full logging:

```
cd /data/openpilot/selfdrive/car/modules/teslaEpasFlasher
sudo ./flash.sh 2>&1 | tee /tmp/flash.log
```

Same milestones as Path A. Watch for "FLASH PROCESS COMPLETED" or "FLASH PROCESS FAILED" at the end. Either way, the entire output is saved to /tmp/flash.log so we can debug.

2B.3 — On failure, copy the log off the 3X to your laptop:

```
scp comma@192.168.43.1:/tmp/flash.log ~/Desktop/
```

Then text Derek the file.

2B.4 — On success, reboot the 3X:

```
sudo reboot
```

STEP 2.5 — WHAT THE FLASHER ACTUALLY DID

If everything completed cleanly, the flasher wrote 4 bytes at each of these 3 firmware addresses, replacing the original instruction with "load 1 into the gate signal":

Firmware address	Original bytes	New bytes	Effect
`0x031750`	`80 ff 74 2b`	`20 56 01 00`	EPB_epasEACAllow always reads 1
`0x031892`	`80 ff 32 2a`	`20 56 01 00`	GTW_epasControlType always reads 1 (ANGLE)
`0x031974`	`80 ff 50 29`	`20 56 01 00`	GTW_epasLDWEnable always reads 1

STEP 3 — VERIFY THE PATCH TOOK

After reflashing, before sending steering commands, verify the rack is now patched. Two methods.

Method A — Quick observable test (preferred)

3A.1 — Cycle the car: Park > fully off > wait 30 seconds > Drive Ready. This forces the rack to reload firmware from flash.

3A.2 — On your laptop with SYS TEC connected, run can_sniffer.py. Confirm 0x370 traffic.

3A.3 — Run move.py with a small angle:

```
python move.py 5
```

3A.4 — Watch the wheel and the sniffer's eacStatus decode (page 2 of can_sniffer.py output):

- eacStatus goes INHIBITED > AVAILABLE > ACTIVE within 1-2 seconds → **patched**
- Wheel turns to about +5 deg → **patched and working**
- eacStatus stays INHIBITED forever → **patch did NOT take, see Step 4**

Method B — Read back the patched bytes (definitive)

If Method A is ambiguous, this method extracts the firmware bytes at the 3 patch addresses without writing anything. Run on the 3X:

```
ssh comma@192.168.43.1
cd /data/openpilot/selfdrive/car/modules/teslaEpasFlasher
PYTHONPATH=/data/openpilot ./patch.py --extract-only
```

This re-reads the live rack firmware (does NOT write). Output file is epas-firmware-0x7000-0x45fff.bin. Then check the 3 patch addresses:

```
python3 -c "
with open('epas-firmware-0x7000-0x45fff.bin','rb') as f: fw = f.read()
for addr in [0x031750, 0x031892, 0x031974]:
  idx = addr - 0x7000
  print(f'{hex(addr)}: {fw[idx:idx+4].hex()}')
"
```

Each address should print **20560100** if patched. If you see 80ff742b / 80ff322a / 80ff5029 the patch did not take.

STEP 4 — POST-FLASH BENCH TEST

Same as the original bench procedure, but now the rack should actually move:

4.1 — Front wheels off ground. Tesla in Accessory or Drive Ready. SYS TEC plugged into OBD-II.

4.2 — Run can_sniffer.py. Confirm "TESLA CHASSIS CAN CONFIRMED" and 0x370 traffic.

4.3 — Run move.py with target = 0:

```
python move.py 0
```

Wheel should not move (already centered) but you may hear a faint click as the rack engages. On the sniffer, eacStatus should transition INHIBITED > AVAILABLE > ACTIVE.

4.4 — Press Ctrl-C. The script sends 5 disengage frames. eacStatus should drop back to AVAILABLE or INHIBITED.

4.5 — Test direction. Run move.py with +5 then with -5. Note which way the wheel turns and which direction is positive. Stop after each. Write down the sign convention.

4.6 — Larger angle test. Run move.py 30. Wheel should turn smoothly to roughly +30 degrees. Ctrl-C. Then -30. Then 0.

4.7 — Switch to GUI tool (tesla_steering_test.py). Click CONNECT, wait for EPAS LINK OK in green, click ENGAGE. Confirm EAC Status reads ACTIVE. Test E-STOP, ESC key, and slider.

STEP 5 — TROUBLESHOOTING MATRIX

Symptom on left, most likely cause and fix on the right. Try fixes in the order listed.

Symptom	Cause	Fix
eacStatus stays INHIBITED after reflash AND verify shows un-patched bytes	Tesla shipped your rack with a different stock firmware version that does not match Greg's MD5.	Read back full firmware (Method B), email Derek the .bin file. We may need a custom patch.
eacStatus stays INHIBITED after reflash but verify shows correct patched bytes	Rack rebooted but did not reload patched firmware. Caching issue.	Pull rack high-current fuse for 60 seconds (frunk fuse box, big fuse for power steering). Reinstall.
Flasher prints "FLASH PROCESS FAILED" at extract step	CAN bus not reaching rack. OBD-II pins 1/9 not populated, or 3X panda not on chassis CAN bus 0.	Run sniffer first, confirm 0x370 received. Check 3X harness.
Flasher prints "expected MD5 sum mismatch"	Your stock rack firmware version is different from what Greg's tool supports.	STOP. Do not force. Send Derek the actual MD5 you got.
Flasher hangs at "request security access seed"	Rack already in extended diagnostic session from prior attempt, or EPB is contending.	Helper releases brake, wait 30 sec, presses brake again, retry.
Flasher succeeds, then car won't drive afterward	Rack rebooted but BCM didn't re-handshake. Manual steering may feel heavy.	Cycle ignition fully OFF for 60 seconds, then back to Drive Ready. Should reset.
eacStatus = FAULT, eacErrorCode = MIN_SPEED	Patched, but rack thinks car is stationary and refuses to steer at low speed.	Either (a) drive the car >3 mph, or (b) bench-mode inject fake ESP_B 0x155 with vehicleSpeed = 30 km/h.
eacStatus = FAULT, eacErrorCode = HANDS_ON	Driver is touching steering wheel; rack reads applied torque.	Let go of the wheel completely.
eacStatus = FAULT, eacErrorCode = HIGH_ANGLE_RATE_REQ	We commanded too fast a change.	Lower MAX_RATE_DEG_PER_SEC in tesla_steering_test.py from 50 to 25.
eacStatus = FAULT, eacErrorCode = HIGH_ANGLE_REQ	Commanded angle exceeds rack-internal limit at current speed.	Lower HARD_ANGLE_LIMIT_DEG. The rack faults above ±500 deg even at standstill.
move.py runs, no errors, but no 0x488 visible on a second sniffer	TX-side problem: SYS TEC driver, USB cable, or python-can not actually opening adapter.	Reinstall SYS TEC driver. Try a different USB-A port.
GUI shows EAC Status flickering AVAILABLE/INHIBITED rapidly	Counter not incrementing or checksum miscalculated; rack rejects every other frame.	Should not happen with our code. If it does, capture the 0x488 bytes via sniffer and send to Derek.
3X says "Driver Monitoring Camera Malfunctioned"	Unrelated to rack. DM camera not detecting face.	Adjust 3X mounting angle. See main openpilot install guide.
SSH to 3X fails with "connection refused"	SSH not enabled in 3X settings.	On 3X UI: Settings > Toggles > Enable SSH. Use comma user, default password 'comma'.

RESTORE TO STOCK (if you ever need to undo)

If you sell the car, return it to a dealer, or just want to remove the patch:

Path A — UI: open Settings > BogGyver Tesla settings > tap **Restore EPAS**.
Path B — SSH:

```
ssh comma@192.168.43.1
cd /data/openpilot/selfdrive/car/modules/teslaEpasFlasher
sudo ./restore.sh
```

Same brake-press requirement. After successful restore, the rack returns to stock behavior and ignores 0x488 commands. Manual steering unaffected.

SOURCES (everything verified)

• gregjhogan/tesla-pre-ap-epas-patch README and patch.py source
• BogGyver/openpilot tesla_unity_releaseC3 branch, selfdrive/car/modules/teslaEpasFlasher/ (patch.py, flash.sh, restore.sh, info.sh, flashTeslaEPAS, epas-bootloader-0x3ff7000-0x3ffacbd.bin)
• BogGyver/openpilot tesla_unity_releaseC3 branch, selfdrive/car/modules/qt/tsettings.cc line 264 (the "Flash EPAS" button wiring)
• commaai/opendbc tesla_can.dbc (eacStatus and eacErrorCode definitions)